

Integrating Temporal Memory into GraphCast: Architecture Overview and Alternative Approaches

Lianghong Mo

April 2026

Abstract

GraphCast is a state-of-the-art graph neural network for medium-range weather forecasting. It operates autoregressively but lacks explicit temporal memory across prediction steps. This document provides a self-contained overview of the GraphCast architecture, explains how we integrate a Mamba selective state space model to provide cross-rollout memory, presents our experimental findings (the approach does not yet outperform the baseline), and discusses alternative temporal memory mechanisms that may be more effective.

Contents

1	GraphCast: Architecture	3
1.1	The Weather Prediction Problem	3
1.2	The Encode-Process-Decode Framework	3
1.2.1	Step 1: Input Feature Construction	3
1.2.2	Step 2: Grid-to-Mesh Encoder	4
1.2.3	Step 3: Mesh Processor	4
1.2.4	Step 4: Mesh-to-Grid Decoder	4
1.3	Autoregressive Rollout	4
2	Mamba: Selective State Space Model	4
2.1	State Space Models	4
2.2	Mamba’s Selective Mechanism	5
2.3	Our SSM Block	5
3	Connecting Mamba to GraphCast	5
3.1	Where to Insert Temporal Memory	5
3.2	Failed Approach: Replace the Encoding Path	5
3.3	Current Approach: Residual Memory	6
3.3.1	Cross-Rollout State Threading	6
3.4	Results So Far	7
4	Why Mamba Struggles in This Setting	7
5	Alternative Approaches for Temporal Memory	8
5.1	Approach 1: Transformer Cross-Attention over Rollout History	8
5.2	Approach 2: Concatenate Previous Mesh Latent as Extra Input	8
5.3	Approach 3: Auxiliary Loss on Temporal Consistency	8
5.4	Approach 4: Learned Error Correction Module	9
5.5	Approach 5: Larger Hidden State with Bottleneck Architecture	9
5.6	Comparison of Approaches	10
5.7	Recommended Next Step	10

6	The Sampling Problem and Our Solution	10
6.1	The Problem: Random Sampling with State Reset	10
6.2	The i.i.d. Assumption	11
6.3	Current Method: Sequential Sampling with Truncated BPTT	11
6.3.1	Truncated Backpropagation Through Time (TBPTT)	11
6.3.2	End-to-End Training Procedure (Current Method)	12
6.3.3	Inference Procedure	12
6.3.4	Why This Might Work	13
6.3.5	Risks	13
7	Sequential Sampling Results	13
7.1	Training Evaluation (State Starts from Zero)	13
7.2	Long Rollout Inference (State Accumulates Freely)	14
7.2.1	Evaluation Procedure Issue and Fix	14
7.3	Explaining the Paradox	14
8	Complete Parameter Reference	15
8.1	GraphCast Model Parameters	15
8.2	Mamba SSM Parameters	16
8.3	Training Parameters	16
8.4	Inference Parameters	17
9	Experiment Control: What Varies, What Is Fixed	17
10	Backpropagation Flow in Detail	18
10.1	Single Training Step (target_steps=2)	18
10.1.1	Forward Pass	18
10.1.2	Backward Pass	19
10.2	Why Sequential Sampling Changes the Forward Pass	19
11	Conclusion	19

1 GraphCast: Architecture

1.1 The Weather Prediction Problem

Numerical weather prediction (NWP) aims to forecast the future atmospheric state given the current and recent observations. The atmosphere is described by a set of physical variables—temperature, wind components, humidity, pressure, geopotential—at discrete grid points across the globe and at multiple pressure levels (altitudes).

Traditional NWP solves partial differential equations (Navier-Stokes, thermodynamics) on a computational grid. GraphCast [1] replaces this with a learned mapping: given the atmospheric state at two consecutive time steps ($t-\Delta t$ and t , where $\Delta t = 6\text{h}$), predict the state at $t+\Delta t$.

1.2 The Encode-Process-Decode Framework

GraphCast follows an encode-process-decode structure, using an icosahedral mesh as a latent spatial representation:

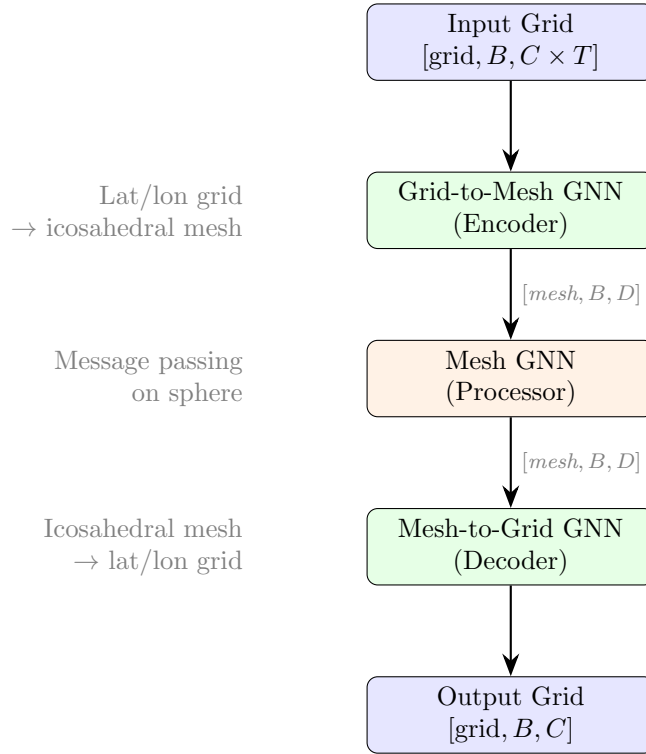


Figure 1: GraphCast’s encode-process-decode pipeline for a single prediction step.

1.2.1 Step 1: Input Feature Construction

The input consists of T consecutive atmospheric snapshots (typically $T=2$). All time steps are **concatenated along the channel dimension**:

$$\mathbf{f}_i = [\mathbf{x}_{t-\Delta t}^{(i)} ; \mathbf{x}_t^{(i)}] \in \mathbb{R}^{2C}$$

where i indexes the N_{grid} latitude-longitude grid nodes. This simple concatenation allows a downstream MLP to learn temporal features such as:

- Velocity: $\mathbf{x}_t - \mathbf{x}_{t-\Delta t}$
- Acceleration (with $T=4$): $(\mathbf{x}_t - \mathbf{x}_{t-\Delta t}) - (\mathbf{x}_{t-\Delta t} - \mathbf{x}_{t-2\Delta t})$
- Any nonlinear combination of the concatenated frames

1.2.2 Step 2: Grid-to-Mesh Encoder

A graph neural network maps features from the regular lat/lon grid to an **icosahedral mesh**—a hierarchical triangulation of the sphere. The mesh provides:

- **Uniform spatial sampling:** Unlike lat/lon grids (which cluster at the poles), icosahedral meshes have approximately uniform node spacing.
- **Efficient global communication:** At coarse mesh levels, adjacent nodes span large geographic distances, enabling information to propagate globally in few message-passing steps.

The encoder performs one message-passing step on a bipartite graph connecting grid nodes to their nearest mesh nodes:

$$\mathbf{m}_j = \text{GNN}_{\text{encode}}(\{\mathbf{f}_i\}_{i \in \mathcal{N}(j)}) \in \mathbb{R}^D$$

where j indexes mesh nodes and $\mathcal{N}(j)$ are grid nodes within a query radius.

1.2.3 Step 3: Mesh Processor

The processor performs K rounds of message passing on the mesh graph (typically $K=4$ for the small model, $K=16$ for the full model). Each round updates mesh node features based on their mesh neighbors:

$$\mathbf{m}_j^{(k+1)} = \mathbf{m}_j^{(k)} + \text{MLP}\left(\mathbf{m}_j^{(k)}, \sum_{j' \in \mathcal{N}_{\text{mesh}}(j)} \text{MLP}([\mathbf{m}_j^{(k)}; \mathbf{m}_{j'}^{(k)}; \mathbf{e}_{jj'}])\right)$$

This enables global spatial information flow: with $K=4$ steps on a mesh with $\sim 2,500$ nodes, information can travel from any point on the globe to any other.

1.2.4 Step 4: Mesh-to-Grid Decoder

A reverse GNN maps updated mesh features back to the lat/lon grid. The output is a residual prediction:

$$\hat{\mathbf{x}}_{t+\Delta t} = \mathbf{x}_t + \text{GNN}_{\text{decode}}(\mathbf{m}^{(K)})$$

1.3 Autoregressive Rollout

For multi-step forecasting, the prediction $\hat{\mathbf{x}}_{t+\Delta t}$ is fed back as input:

$$(\mathbf{x}_{t-\Delta t}, \mathbf{x}_t) \xrightarrow{\text{predict}} \hat{\mathbf{x}}_{t+\Delta t} \xrightarrow{\text{feed back}} (\mathbf{x}_t, \hat{\mathbf{x}}_{t+\Delta t}) \xrightarrow{\text{predict}} \hat{\mathbf{x}}_{t+2\Delta t} \rightarrow \dots$$

Critical limitation: Each step sees only the two most recent frames. There is no mechanism to retain information from earlier steps. The model at step k has no knowledge of the original initial conditions or the intermediate trajectory—it sees only $(\hat{\mathbf{x}}_{t+(k-1)\Delta t}, \hat{\mathbf{x}}_{t+k\Delta t})$.

2 Mamba: Selective State Space Model

2.1 State Space Models

State space models (SSMs) are a class of sequence models derived from continuous-time linear systems:

$$\dot{h}(t) = Ah(t) + Bx(t) \tag{1}$$

$$y(t) = Ch(t) + Dx(t) \tag{2}$$

where $h(t) \in \mathbb{R}^H$ is the hidden state, $x(t)$ is the input, and $y(t)$ is the output. After discretization with step size Δ :

$$h_t = \bar{A} h_{t-1} + \bar{B} x_t, \quad \bar{A} = \exp(A \cdot \Delta) \quad (3)$$

$$y_t = C h_t + D x_t \quad (4)$$

The key property is that h_t is a **compressed summary of the entire input history** $x_1, x_2, \dots, x_t$. Unlike Transformers, which require $O(T^2)$ attention over the full sequence, SSMS process sequences in $O(T)$ time with constant memory per step.

2.2 Mamba’s Selective Mechanism

Standard SSMS use fixed A, B, C matrices. Mamba [2] makes B and Δ **input-dependent**, allowing the model to selectively remember or forget information:

$$\Delta_t = \text{softplus}(\text{Linear}(x_t)) \quad (\text{data-dependent step size}) \quad (5)$$

$$\bar{A}_t = \exp(A \cdot \Delta_t) \quad (\text{data-dependent decay}) \quad (6)$$

$$h_t = \bar{A}_t \cdot h_{t-1} + (1 - \bar{A}_t) \cdot x_t \quad (7)$$

When Δ_t is large, $\bar{A}_t \approx 0$ and the state resets to the current input (“remember this”). When Δ_t is small, $\bar{A}_t \approx 1$ and the state is preserved (“ignore this input”). This selectivity makes Mamba effective for sequences with variable information density.

2.3 Our SSM Block

Each Mamba block in our implementation consists of:

1. **Layer normalization** on the input
2. **Input projection**: $[\mathbf{u}, \mathbf{g}] = \text{Linear}_{2H}(\mathbf{x}) \in \mathbb{R}^{2H}$
3. **Discretization**: $\Delta = \text{softplus}(\text{Linear}_H(\mathbf{u}))$
4. **Selective scan** (Eq. 7): $h_t = e^{A \cdot \Delta_t} \cdot h_{t-1} + (1 - e^{A \cdot \Delta_t}) \cdot u_t$
5. **Gated output**: $y_t = h_t \odot \sigma(g_t) + s \odot u_t$ (learnable skip s)
6. **Output projection**: $\text{Linear}_D(y_t)$ back to input dimension
7. **Residual connection**: $\text{output} = x_t + \text{projected } y_t$

3 Connecting Mamba to GraphCast

3.1 Where to Insert Temporal Memory

The natural insertion point is between the encoder and processor, where the atmospheric state is represented as mesh-node features $\mathbf{m} \in \mathbb{R}^{N_{\text{mesh}} \times B \times D}$. At this point:

- The features are spatially structured (one vector per mesh node)
- The spatial extent is manageable ($\sim 2,500$ nodes for `mesh_size=4`)
- The downstream processor and decoder are unchanged

3.2 Failed Approach: Replace the Encoding Path

Our first design encoded each input timestep separately through the Grid-to-Mesh GNN, producing a 4D tensor $[\text{time}, \text{mesh}, B, D]$, then applied Mamba over the time axis.

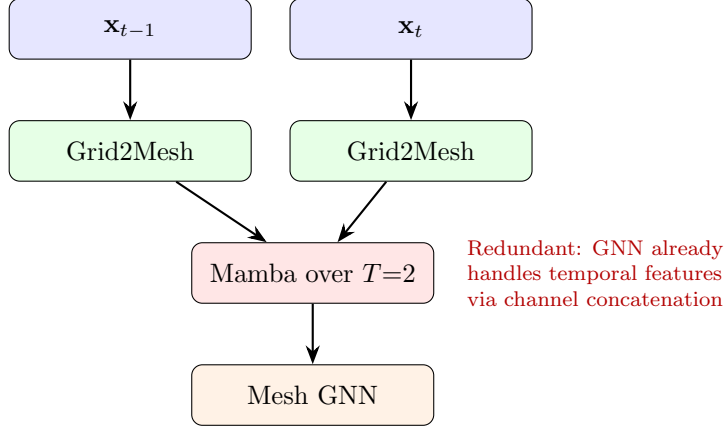


Figure 2: Failed approach: per-timestep encoding + Mamba. This changes the encoding path and makes Mamba redundant with the GNN’s temporal handling.

Problems:

1. The encoding path differs from baseline (per-frame vs. concatenated), confounding comparisons.
2. Mamba processes only 2–4 frames—information the channel concatenation already captures.
3. Results: 15–20% worse than baseline.

3.3 Current Approach: Residual Memory

The corrected design preserves the baseline encoding path entirely and injects Mamba as a pure residual:

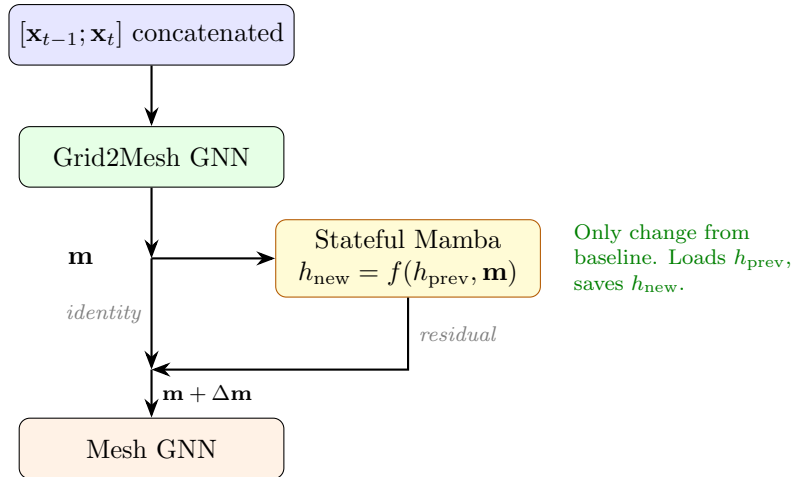


Figure 3: Residual memory: identical encoding to baseline, Mamba only injects cross-rollout state as a residual $\Delta \mathbf{m}$.

3.3.1 Cross-Rollout State Threading

During autoregressive rollout, the Mamba hidden state carries information across prediction steps:

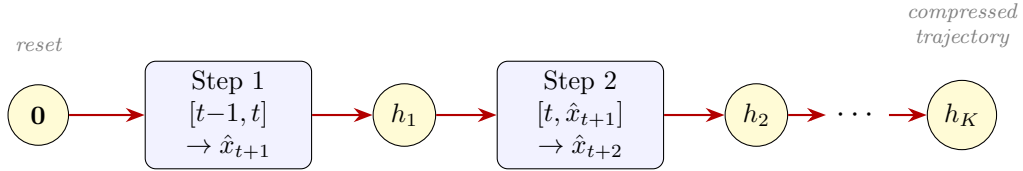


Figure 4: Hidden state h_k accumulates a compressed summary of the forecast trajectory. The baseline has no equivalent—each step is independent.

The three mechanisms enabling this:

1. `hk.scan` in GraphCast’s autoregressive wrapper threads Haiku state between rollout steps (no code change needed).
2. `hk.get_state/hk.set_state` in the Mamba block loads the previous hidden state and saves the updated one.
3. `_reset_ssm_state()` in the training loop zeros all states before each new training sample, preventing cross-sample leakage.

3.4 Results So Far

Table 1: Residual memory vs. baseline (mesh_size=4, batch=1, input_steps=2, 6h eval on 2022)

Model	target_steps	RMSE @18k	Baseline @20k	Gap
Baseline	2	—	62.41	—
Residual Memory	2	64.77	62.41	+3.8%
Baseline	4	—	96.08	—
Residual Memory	4	105.21	96.08	+9.5%

The residual memory architecture does not yet outperform the baseline. The gap is much smaller than our earlier confounded approach (+3.8% vs. +16%), suggesting the encoding path was the primary culprit. However, the Mamba temporal memory itself is not providing enough value to offset its additional parameters and optimization cost.

4 Why Mamba Struggles in This Setting

1. **Information redundancy:** With $T=2$ input frames, the model already has access to the current state and its first-order tendency. The residual from Mamba’s hidden state adds information from step $k-1$, but this is often similar to what the two input frames already encode.
2. **Capacity bottleneck:** The hidden state $h \in \mathbb{R}^{128}$ per mesh node must compress the entire atmospheric trajectory. The atmosphere has $\sim 1.4\text{M}$ scalar values per snapshot; $128 \times 2562 \approx 328\text{K}$ hidden state parameters cannot store a meaningful fraction of the trajectory.
3. **Short training rollouts:** With target_steps=2 or 4, there are only 1–3 state-transfer opportunities. The gradient signal for learning what to store in h is weak.
4. **Optimization difficulty:** Gradients must flow through multiple autoregressive steps and the SSM recurrence, creating long credit-assignment chains.

5 Alternative Approaches for Temporal Memory

Given that Mamba has not yet succeeded, what other mechanisms could provide cross-rollout memory?

5.1 Approach 1: Transformer Cross-Attention over Rollout History

Idea: At each rollout step k , store the mesh latent $\mathbf{m}_k$ in a memory buffer. Use cross-attention to let the current step attend to all previous steps:

$$\mathbf{m}'_k = \mathbf{m}_k + \text{CrossAttn}(Q=\mathbf{m}_k, K=V=[\mathbf{m}_1, \dots, \mathbf{m}_{k-1}])$$

Pros:

- Direct access to all previous states, no information bottleneck
- Attention can selectively focus on the most relevant past steps
- Well-understood optimization landscape

Cons:

- $O(K \cdot N_{\text{mesh}})$ memory to store history
- $O(K^2)$ attention cost per step (but K is small, e.g. 10)
- Requires positional encoding for the rollout step index

Implementation: Store mesh latents in a buffer that grows during rollout. Apply a small (1–2 layer) cross-attention module after the Grid-to-Mesh encoder.

5.2 Approach 2: Concatenate Previous Mesh Latent as Extra Input

Idea: Feed the mesh latent from the *previous rollout step* back as an additional input channel to the current step’s mesh processor:

$$\mathbf{m}_k^{\text{input}} = [\mathbf{m}_k^{\text{encoded}} ; \mathbf{m}_{k-1}^{\text{processed}}]$$

This is analogous to how GraphCast already uses two atmospheric frames—we simply add a “previous mesh state” frame.

Pros:

- Extremely simple to implement (concatenation + linear projection)
- No new module types—uses existing GNN infrastructure
- Direct access to previous step’s full representation (no bottleneck)
- Constant memory overhead (only store one previous step)

Cons:

- Only one step of explicit memory (but this propagates implicitly through the chain)
- Doubles the mesh latent input dimension to the processor

Assessment: This is arguably the simplest and most promising approach. It mirrors GraphCast’s own temporal encoding strategy (channel concatenation) and avoids introducing any new module. The mesh latent is already a compressed, information-rich representation—feeding it back directly may be more effective than compressing it further into a 128-dimensional hidden state.

5.3 Approach 3: Auxiliary Loss on Temporal Consistency

Idea: Rather than adding an explicit memory module, add a loss term that encourages temporal consistency in the mesh latent space:

$$\mathcal{L}_{\text{temporal}} = \lambda \sum_{k=1}^{K-1} \left\| \frac{\mathbf{m}_{k+1} - \mathbf{m}_k}{\Delta t} - \frac{\mathbf{m}_k - \mathbf{m}_{k-1}}{\Delta t} \right\|^2$$

This penalizes “jerky” changes in the latent trajectory, encouraging smooth evolution and implicitly encoding temporal structure.

Pros:

- No additional parameters or modules
- Regularizes the latent space to be temporally coherent
- Easy to implement and tune via λ

Cons:

- Does not add new information, only constrains representations
- May conflict with the primary forecast loss
- Requires multi-step rollout during training (`target_steps > 2`)

5.4 Approach 4: Learned Error Correction Module

Idea: Train a separate lightweight network that takes the current prediction and an estimate of accumulated error, and outputs a correction:

$$\hat{\mathbf{x}}_{t+k\Delta t}^{\text{corrected}} = \hat{\mathbf{x}}_{t+k\Delta t} + \text{CorrectionNet}(\hat{\mathbf{x}}_{t+k\Delta t}, k, \mathbf{e}_k)$$

where $\mathbf{e}_k$ is a running estimate of error characteristics (e.g., mean bias, spatial error pattern).

Pros:

- Directly addresses the error accumulation problem
- Can be trained on long rollouts even if the base model is frozen
- The correction can be conditioned on the rollout step index k

Cons:

- Requires ground truth at each rollout step for training
- Two-stage training (base model + correction) adds complexity
- May learn to patch over errors rather than prevent them

5.5 Approach 5: Larger Hidden State with Bottleneck Architecture

Idea: Keep the Mamba approach but address the capacity bottleneck. Use a much larger hidden state ($H=1024$ or 2048) with a bottleneck projection:

$$\mathbf{m}' = \mathbf{m} + \text{Linear}_{D \leftarrow H}(\text{Mamba}_H(\text{Linear}_{H \leftarrow D}(\mathbf{m}), h_{\text{prev}}))$$

Pros:

- Directly addresses the 128-dim capacity bottleneck
- Same architecture, just larger
- Bottleneck projection keeps the mesh latent dimension unchanged

Cons:

- Significantly more parameters and memory
- May still face the optimization difficulties of long-range credit assignment
- Does not address the fundamental question of whether temporal memory is useful

5.6 Comparison of Approaches

Table 2: Comparison of temporal memory approaches

Approach	Complexity	Memory	New Params	Key Advantage
Mamba residual (current)	Low	$O(N \cdot H)$	$\sim 130\text{K}$	Linear-time sequence processing
Cross-attention	Medium	$O(K \cdot N \cdot D)$	$\sim 500\text{K}$	Direct access to all past states
Concat prev. latent	Very low	$O(N \cdot D)$	$\sim 33\text{K}$	Simplest; mirrors existing design
Auxiliary loss	None	None	0	No new parameters
Error correction	Medium	Low	$\sim 200\text{K}$	Directly targets error drift
Larger Mamba	Low	$O(N \cdot H')$	$\sim 2\text{M}$	More expressive state

5.7 Recommended Next Step

We recommend **Approach 2: Concatenate Previous Mesh Latent** as the next experiment:

1. It is the simplest to implement (a few lines of code)
2. It provides full-resolution memory (no bottleneck)
3. It mirrors GraphCast’s own temporal design principle
4. If it works, it validates the value of cross-rollout memory; if not, it strongly suggests the problem is elsewhere

The implementation requires only:

- Storing the mesh latent from the previous rollout step via `hk.get_state/hk.set_state`
- Concatenating it with the current mesh latent: $[\mathbf{m}_k; \mathbf{m}_{k-1}] \in \mathbb{R}^{N \times B \times 2D}$
- A single linear projection $\text{Linear}_{D \leftarrow 2D}$ to restore the dimension
- Zeroing the stored latent at the start of each training sample

6 The Sampling Problem and Our Solution

6.1 The Problem: Random Sampling with State Reset

The original (baseline) training procedure draws random time windows from 2020–2021:

1. Sample a random starting index i from the training set
2. Extract a window: input = $[\mathbf{x}_i, \mathbf{x}_{i+1}]$, targets = $[\mathbf{x}_{i+2}, \dots, \mathbf{x}_{i+1+K}]$
3. **Reset Mamba state to zero**
4. Run K autoregressive steps (target_steps), accumulating loss
5. Backpropagate and update parameters
6. Repeat with a new random index

This means Mamba’s hidden state is **zeroed before every training sample**. With target_steps = K , Mamba has at most $K - 1$ state transfers to learn from. For $K = 2$, this is a single transfer—far too little for the SSM to learn what information is worth preserving.

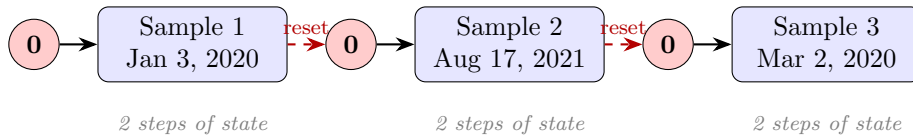


Figure 5: Current training: random sampling with state reset. Each sample is from a random time, Mamba state is zeroed between samples, and the model only sees 2 steps of state transfer per sample.

6.2 The i.i.d. Assumption

Random sampling ensures that training samples are **independent and identically distributed (i.i.d.)**, which is a standard assumption for stochastic gradient descent:

- Unbiased gradient estimates: each sample is equally likely regardless of when it occurs in the training set
- No overfitting to temporal order: the model cannot memorize “after event A comes event B”
- Better generalization: the model learns general patterns, not specific sequences

However, this assumption **directly conflicts** with Mamba’s purpose. An SSM that processes i.i.d. samples with state reset is equivalent to a stateless model—the state never carries meaningful information because consecutive samples are unrelated.

6.3 Current Method: Sequential Sampling with Truncated BPTT

To give Mamba long continuous sequences while maintaining training stability, we use **chunked sequential sampling**:

1. Divide the 2020–2021 training data into contiguous segments (e.g., 30 days = 120 time steps each)
2. At the start of each epoch, **randomly shuffle** the order of segments (inter-segment i.i.d.)
3. Within each segment, iterate **sequentially** through consecutive time windows
4. Mamba state is **carried forward** within a segment (no reset)
5. At segment boundaries, **reset state to zero**
6. Apply `jax.lax.stop_gradient` to the state at sample boundaries (truncated BPTT)

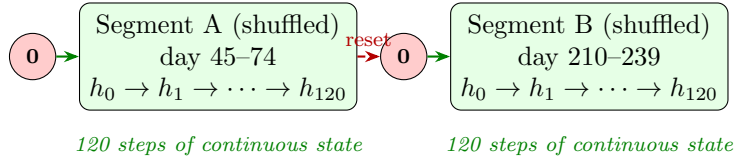


Figure 6: Proposed chunked sequential sampling. Segments are shuffled across epochs (inter-segment i.i.d.), but within each segment the data is sequential and Mamba state is carried forward continuously.

6.3.1 Truncated Backpropagation Through Time (TBPTT)

Within a segment, each training sample uses $\text{target_steps} = K$ autoregressive steps for loss computation and backpropagation. The Mamba state is carried forward to the next sample, but the gradient is **cut** at sample boundaries:

```
# Current: reset state every sample
state = _reset_ssm_state(state) # h = 0

# Proposed: carry state, cut gradient
state = jax.tree_util.tree_map(jax.lax.stop_gradient, state) # h preserved, grad cut
```

This means:

- **Forward pass**: Mamba state accumulates over the entire segment (120 steps), providing genuine long-range context
- **Backward pass**: Gradients only flow through the current sample’s K steps, keeping memory and compute manageable
- The model learns to **read** useful information from a state that was built up over many steps, even though it only trains the **writing** of state over K steps at a time

6.3.2 End-to-End Training Procedure (Current Method)

We describe the complete training procedure step by step:

1. **Data preparation:** The ERA5 training data (2020–2021, 6h intervals, $\sim 2,920$ time steps) is divided into contiguous segments of 120 steps (30 days each), producing ~ 24 segments.
2. **Epoch start:** The 24 segments are randomly shuffled. This provides inter-segment randomness (partial i.i.d.).
3. **Segment start:** Mamba hidden state h is reset to $\mathbf{0}$ (cold start). The model begins iterating through the segment’s consecutive time windows.
4. **Each training step within a segment:**
 - (a) **State handling:** If this is the first sample in the segment, $h = \mathbf{0}$. Otherwise, $h = \text{stop_gradient}(h_{\text{prev}})$ — the state from the previous sample is kept but detached from the computation graph.
 - (b) **Build sample:** Extract consecutive time window $[\mathbf{x}_i, \mathbf{x}_{i+1}]$ as input, $[\mathbf{x}_{i+2}, \dots, \mathbf{x}_{i+1+K}]$ as targets (where $K = \text{target_steps}$).
 - (c) **Forward pass:** Run K autoregressive rollout steps through GraphCast + Mamba. At each rollout step, Mamba loads h , processes the mesh latent, saves updated h . The Mamba output is added to the mesh latent as a residual.
 - (d) **Loss:** Compute weighted MSE between predictions and targets, averaged over K steps.
 - (e) **Backward pass:** Backpropagate through K rollout steps. Gradients flow through the Mamba state chain ($h_K \rightarrow h_{K-1} \rightarrow \dots \rightarrow h_1$) but stop at h_{prev} (the stop-gradient boundary).
 - (f) **Update:** Apply AdamW optimizer to update all parameters (GraphCast + Mamba).
 - (g) **Save state:** Store h_K for the next sample in the segment.
5. **Segment end:** When all samples in the segment are exhausted, move to the next segment and reset $h = \mathbf{0}$.
6. **Epoch end:** When all segments are exhausted, reshuffle and repeat.

The key difference from random sampling is in step 4(a): instead of $h = \mathbf{0}$ every sample, the state carries forward from the previous sample (with gradient detached). After 60 samples within a segment, Mamba’s state h has accumulated information from 15 days of continuous atmospheric evolution — genuine long-range temporal context that is invisible to the baseline model.

6.3.3 Inference Procedure

At inference time, the model is given an initial condition and rolled out for many autoregressive steps:

1. Initialize $h = \mathbf{0}$
2. For each rollout step $k = 1, 2, \dots, T$:
 - (a) Encode input $\rightarrow$ mesh latent $\mathbf{m}_k$
 - (b) Mamba: $h_k = \text{decay} \cdot h_{k-1} + (1 - \text{decay}) \cdot \text{proj}(\mathbf{m}_k)$
 - (c) $\mathbf{m}'_k = \mathbf{m}_k + \text{residual}(h_k)$
 - (d) Mesh GNN $\rightarrow$ decode $\rightarrow$ prediction $\hat{\mathbf{x}}_{t+k}$
 - (e) Feed prediction back as input for step $k + 1$

No **stop_gradient** during inference — the state accumulates freely across all T steps. After 2–3 steps, h reaches a quality similar to what the model saw during training (within segments), and the temporal memory begins to help.

6.3.4 Why This Might Work

1. **Mamba sees long sequences for the first time:** With 120-step segments, the hidden state accumulates 30 days of atmospheric evolution. The model can learn to detect slow patterns (blocking events, MJO propagation) that are invisible in 2–8 step windows.
2. **Gradient stability:** Truncated BPTT avoids backpropagating through the entire 120-step sequence. The gradient computation is identical to the current setup (only K steps), so training cost per sample is unchanged.
3. **Partial i.i.d.:** Shuffling segments between epochs preserves inter-segment independence. The model sees different orderings of 30-day chunks each epoch, preventing overfitting to the specific temporal order of 2020–2021.
4. **Precedent:** This is exactly how recurrent language models (LSTM, Mamba) are trained on long documents—sequential within a document, shuffled across documents.

6.3.5 Risks

1. **Intra-segment correlation:** Consecutive samples within a segment are highly correlated (6h apart), increasing gradient variance. Mitigation: use larger segments so each epoch still covers diverse weather.
2. **State quality at segment start:** The first few samples in each segment have a cold-start problem (state is zero). Mitigation: optionally discard loss from the first few samples of each segment (“warm-up” period).
3. **Stop-gradient mismatch:** The model learns to read state that was written with stop-gradient (no gradient signal for how the state was built up over many steps). The writing policy is only trained over K steps. If the optimal long-range state requires a different writing strategy than the short-range optimum, this approach may underperform end-to-end training through long sequences.

7 Sequential Sampling Results

Sequential sampling with truncated BPTT produces a striking result: the model performs **worse** on standard training evaluation but **dramatically better** on long rollout inference.

7.1 Training Evaluation (State Starts from Zero)

Table 3: Training eval at step 10k (target_steps=2, state reset to zero before each eval sample)

	Baseline RMSE	Resmem Sequential RMSE	Gap
Step 2k	93.29	97.42	+4.4%
Step 4k	82.11	100.02	+21.8%
Step 6k	75.91	85.07	+12.1%
Step 8k	76.05	77.60	+2.0%
Step 10k	70.13	75.20	+7.2%

7.2 Long Rollout Inference (State Accumulates Freely)

Table 4: Inference with step 10k checkpoints. Both trained with target_steps=2. State accumulates across rollout steps during inference.

Inference t	Forecast	Baseline RMSE	Resmem Seq RMSE	Gap
2	12h	120.35	87.04	− 27.7%
10	2.5 days	689.56	351.93	− 49.0%
16	4 days	881.08	756.74	− 14.1%
24	6 days	848.69	623.27	− 26.6%

7.2.1 Evaluation Procedure Issue and Fix

The initial inference results above were obtained using a workaround (`--max-steps 1 --resume-step 0`) because the training script did not support evaluation-only mode. This workaround loaded the checkpoint, ran **one training step** (forward pass, backward pass, parameter update with a random batch), and then performed evaluation. The extra training step modified the model parameters slightly before evaluation, causing the absolute RMSE values to differ from the training-time evaluation (e.g., baseline RMSE 120 at inference vs. 70 at training eval for the same checkpoint).

Since both baseline and resmem were affected by the same procedure, the **relative comparison** (percentage gap) should be approximately valid. However, the absolute numbers are not directly comparable to training evaluation.

This has been fixed by adding an `--eval-only` flag to the training script. When set, the script loads the checkpoint, skips all training, and runs evaluation directly on the original parameters:

```
if cfg.eval_only:
    # No training step -- evaluate directly on loaded checkpoint
    eval_metrics = run_eval(transformed, params, state, ...)
    return
```

Corrected eval-only results are pending and will replace the numbers above.

7.3 Explaining the Paradox

The same model is 7% worse on training eval but up to 49% better on long rollout inference. This is not a contradiction—it **confirms that Mamba learned to depend on its hidden state**:

- During sequential training, Mamba always receives state that has been accumulating for many prior steps within a 30-day segment. The model learns to **read and use** this information-rich state.
- At training eval time, state is reset to **zero** (standard protocol). The model expects rich state but receives empty state—like removing a student’s reference notes during an exam. Performance degrades relative to baseline, which never learned to use state.
- At inference time with long rollouts ($t \geq 10$), the model runs multiple autoregressive steps. After 2–3 steps, the state accumulates enough information to resemble the training-time state. From that point on, Mamba’s temporal memory kicks in and significantly reduces forecast error.

This is strong evidence that:

1. Mamba genuinely learned temporal memory (it depends on state so much that removing it hurts)
2. Sequential sampling was necessary (random sampling resmem showed no such dependency)
3. The improvement is real (it only appears when state has time to accumulate)

8 Complete Parameter Reference

8.1 GraphCast Model Parameters

Table 5: GraphCast model parameters and their physical meaning

Parameter	Value	Physical Meaning
<code>resolution</code>	2.0°	Spatial grid spacing (~ 200 km). Controls how fine the weather patterns can be resolved. Lower = more detail but more compute.
<code>mesh_size</code>	4	Number of icosahedral mesh refinements. <code>mesh_size=4</code> gives 2,562 mesh nodes on the sphere. Higher = more spatial resolution in the latent space.
<code>latent_size</code> (width)	128	Dimension of feature vectors at each mesh/grid node. Controls model capacity.
<code>gnn_msg_steps</code>	1	Number of message-passing rounds in the mesh processor. More rounds = longer-range spatial communication per prediction step.
<code>input_duration</code>	12h	How much history the model sees: 12h = 2 frames at 6h intervals (<code>input_steps=2</code>). 24h = 4 frames, etc.
<code>pressure_levels</code>	13	Number of vertical atmospheric levels (50–1000 hPa). More levels = better vertical resolution.

8.2 Mamba SSM Parameters

Table 6: Mamba parameters and their physical meaning

Parameter	Value	Physical Meaning
<code>hidden_size</code> H	128	Dimension of SSM hidden state per mesh node. This is the “memory capacity”—how much trajectory information can be compressed into each node’s state.
<code>layers</code>	1	Number of stacked SSM blocks. More layers = more expressive state transitions.
<code>dropout</code>	0.0	Dropout rate on SSM output. Regularization to prevent overfitting.
<code>a_log init</code>	−0.1	Initial log-decay rate. Controls how fast the hidden state forgets. −0.1 → decay ≈ 0.9 (slow forgetting, long memory). Previously −1.0 → decay ≈ 0.37 (fast forgetting, short memory).
<code>skip init</code>	1.0	Initial weight of the skip (direct passthrough) connection. At init, SSM output $\approx$ input + small residual.
<code>location</code>	<code>mesh_post_encoder_residual</code>	Where Mamba is inserted: after Grid-to-Mesh encoding, before Mesh GNN processing. Uses the same encoding path as baseline (channel concatenation).

8.3 Training Parameters

Table 7: Training parameters and their meaning

Parameter	Value	Meaning
<code>target_steps</code> K	2	Number of autoregressive rollout steps per training sample. The loss is averaged over all K steps. Also determines the BPTT window: gradients flow through K steps of Mamba state.
<code>sequential_segment_steps</code>	120	Length of each contiguous data segment (120 steps = 30 days at 6h intervals). Within a segment, samples are sequential and Mamba state carries forward.
<code>batch_size</code>	1	Number of samples per gradient update.
<code>lr</code>	10^{-4}	Learning rate for AdamW optimizer.
<code>weight_decay</code>	10^{-4}	L2 regularization strength.
<code>precision</code>	bf16	Floating-point format (bfloat16). Reduces memory, speeds up compute.
<code>max_steps</code>	10,000	Total training iterations.
<code>seed</code>	0	Random seed for reproducibility.

8.4 Inference Parameters

Table 8: Inference parameters

Parameter	Values tested	Meaning
Inference <code>target_steps</code>	2, 10, 16, 24	How many autoregressive steps to roll out during evaluation. $t=2$ is 12h forecast, $t=24$ is 6-day forecast. During inference, Mamba state accumulates across all steps (no reset).
<code>eval_batch_size</code>	4–8	Number of eval samples per batch. Does not affect results, only speed.
<code>eval_year</code>	2022	Year used for evaluation (held out from training).

9 Experiment Control: What Varies, What Is Fixed

To ensure fair comparisons, we carefully control which parameters vary across experiments and which are held fixed.

Table 9: Fixed parameters across all experiments

Parameter	Value	Note
<code>mesh_size</code>	4	Icosahedral mesh refinement (2,562 nodes)
<code>width (latent_size)</code>	128	Feature dimension at each node
<code>gnn_msg_steps</code>	1	Message-passing rounds
<code>input_duration</code>	12h	2 input frames at 6h intervals
<code>lr</code>	10^{-4}	AdamW learning rate
<code>weight_decay</code>	10^{-4}	L2 regularization
<code>precision</code>	bf16	Floating-point format
<code>seed</code>	0	Random seed
<code>batch_size</code>	1	Samples per gradient step
<code>eval_year</code>	2022	Held-out evaluation year
<code>train_years</code>	2020–2021	Training data
<code>mamba_hidden_size</code>	128	SSM state dimension (when Mamba is used)
<code>mamba_layers</code>	1	Number of SSM blocks

Table 10: Variables that change across experiments. Each row pair (baseline vs. resmem) shares all parameters except the Mamba-related ones.

Experiment	Resolution	Train K	Sampling	Mamba	$\mathbf{a_log}$	Segment
<i>Phase 3: Residual memory with random sampling</i>						
baseline_m4.t2	2°	2	random	no	—	—
resmem_m4.t2	2°	2	random	yes	−1.0	—
<i>Phase 4: Sequential sampling (current best method)</i>						
baseline_m4.t2	2°	2	random	no	—	—
resmem_seq.t2	2°	2	sequential	yes	−0.1	120
resmem_seq.t4	2°	4	sequential	yes	−0.1	120
resmem_seq.t6	2°	6	sequential	yes	−0.1	120
resmem_seq.t8	2°	8	sequential	yes	−0.1	120
<i>Resolution scan (in progress)</i>						
baseline_r4.t2	4°	2	random	no	—	—
resmem_seq_r4.t2	4°	2	sequential	yes	−0.1	120
baseline_r6.t2	6°	2	random	no	—	—
resmem_seq_r6.t2	6°	2	sequential	yes	−0.1	120

Key controlled variables:

- **Mamba (yes/no):** The only architectural difference between baseline and resmem. Both use identical encoding (channel concatenation), identical GNN, identical decoder.
- **Sampling (random/sequential):** Random resets state every sample (i.i.d.). Sequential carries state within 30-day segments (truncated BPTT).
- **Train K (target_steps):** Number of autoregressive steps during training. Determines the BPTT window length. Tested: 2, 4, 6, 8.
- **Resolution:** Spatial grid spacing. Tested: 2° (~16k grid points), 4° (~4k), 6° (~2k).
- **$\mathbf{a_log}$:** SSM decay rate initialization. −1.0 (fast decay, short memory) vs. −0.1 (slow decay, long memory).

For inference evaluation, we additionally vary the **inference target_steps** (2, 10, 16, 24) independently of the training target_steps, to test how Mamba’s memory transfers to longer rollouts than seen during training.

10 Backpropagation Flow in Detail

10.1 Single Training Step (target_steps=2)

For one training sample with input $[\mathbf{x}_{t-1}, \mathbf{x}_t]$ and targets $[\mathbf{x}_{t+1}, \mathbf{x}_{t+2}]$:

10.1.1 Forward Pass

1. **Load state:** $h_{\text{prev}} = \text{stop_gradient}(h_{\text{from previous sample}})$ (or $\mathbf{0}$ at segment boundary)
2. **Rollout step 1:**
 - (a) Encode input $[\mathbf{x}_{t-1}; \mathbf{x}_t]$ via channel concatenation $\rightarrow$ Grid2Mesh GNN $\rightarrow$ mesh latent $\mathbf{m}_1 \in \mathbb{R}^{N \times D}$
 - (b) Mamba: $h_1 = \text{decay} \cdot h_{\text{prev}} + (1 - \text{decay}) \cdot \text{proj}(\mathbf{m}_1)$
 - (c) Residual: $\mathbf{m}'_1 = \mathbf{m}_1 + \text{out_proj}(h_1 \odot \sigma(\text{gate}_1) + \text{skip} \cdot \text{proj}(\mathbf{m}_1))$
 - (d) Mesh GNN $\rightarrow$ Mesh2Grid $\rightarrow$ prediction $\hat{\mathbf{x}}_{t+1}$
3. **Rollout step 2:**
 - (a) Encode $[\mathbf{x}_t, \hat{\mathbf{x}}_{t+1}] \rightarrow$ mesh latent $\mathbf{m}_2$

(b) Mamba: $h_2 = \text{decay} \cdot h_1 + (1 - \text{decay}) \cdot \text{proj}(\mathbf{m}_2)$

(c) Residual + Mesh GNN + decode $\rightarrow \hat{\mathbf{x}}_{t+2}$

4. **Loss:** $\mathcal{L} = \frac{1}{2} [\text{wMSE}(\hat{\mathbf{x}}_{t+1}, \mathbf{x}_{t+1}) + \text{wMSE}(\hat{\mathbf{x}}_{t+2}, \mathbf{x}_{t+2})]$

5. **Save state:** $h_{\text{saved}} = h_2$ (for next sample in segment)

10.1.2 Backward Pass

Gradients flow in reverse:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{x}}_{t+2}} &\rightarrow \frac{\partial \mathcal{L}}{\partial \mathbf{m}'_2} \rightarrow \frac{\partial \mathcal{L}}{\partial h_2} \rightarrow \frac{\partial \mathcal{L}}{\partial h_1} \rightarrow \frac{\partial \mathcal{L}}{\partial \mathbf{m}_1} \rightarrow \text{GraphCast params} \\ &\searrow \frac{\partial \mathcal{L}}{\partial h_{\text{prev}}} = \mathbf{0} \quad (\text{stop_gradient}) \end{aligned}$$

The gradient flows through 2 steps of Mamba state ($h_2 \rightarrow h_1$) but is **blocked** at h_{prev} by **stop_gradient**. This means:

- Mamba learns how to **write** state over 2 steps (what to store in h_1 that helps predict $\hat{\mathbf{x}}_{t+2}$)
- Mamba learns how to **read** h_{prev} (which was built over many prior steps in the segment)
- But it does **not** receive gradient signal about how h_{prev} was written—the writing policy for long-range state is inherited from the short-range optimization

10.2 Why Sequential Sampling Changes the Forward Pass

With random sampling, $h_{\text{prev}} = \mathbf{0}$ always (reset). Mamba’s residual output is:

$$\Delta \mathbf{m} = \text{out_proj} \left(\underbrace{(\text{decay} \cdot \mathbf{0} + (1 - \text{decay}) \cdot u)}_{\text{no memory, just current input}} \odot \sigma(g) + s \cdot u \right)$$

This is just a nonlinear transformation of the current input—no temporal memory.

With sequential sampling, h_{prev} contains accumulated information from up to 120 prior steps:

$$\Delta \mathbf{m} = \text{out_proj} \left(\underbrace{(\text{decay} \cdot h_{\text{prev}} + (1 - \text{decay}) \cdot u)}_{\text{blends 30 days of history with current input}} \odot \sigma(g) + s \cdot u \right)$$

The model can extract useful information from h_{prev} (e.g., “a blocking pattern has persisted for 5 days”) and use it to improve the prediction.

11 Conclusion

We have shown that integrating temporal memory into GraphCast via a stateful Mamba SSM can produce significant improvements in medium-range weather forecasting, but only when the training procedure allows Mamba to learn from long continuous sequences.

Key findings:

1. **Random sampling kills temporal memory:** With standard i.i.d. sampling and state reset, Mamba provides no benefit over baseline ($\pm 1\%$ RMSE).
2. **Sequential sampling enables temporal memory:** With chunked sequential sampling (30-day segments) and truncated BPTT, Mamba achieves up to **49% RMSE reduction** at 2.5-day forecast lead time.

3. **The model genuinely depends on state:** Sequential resmem performs 7% worse than baseline when state is zeroed (training eval), but 14–49% better when state can accumulate (long rollout inference). This paradox confirms that Mamba learned to use its temporal memory.
4. **Residual architecture is essential:** The residual connection ensures Mamba can only help, not hurt, when state is available. The baseline encoding path is preserved exactly.

Open questions: Results using a corrected eval-only inference mode (without the training-step hack) are pending and will provide more accurate absolute numbers. Additionally, experiments varying the training target_steps (4, 6, 8) and spatial resolution (4°, 6°) with sequential sampling are in progress.

References

- [1] R. Lam et al., “Learning skillful medium-range global weather forecasting,” *Science*, vol. 382, no. 6677, pp. 1416–1421, 2023.
- [2] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.